

Universidad Nacional Autónoma de México

Facultad de Ciencias (2026-1)



Computación Concurrente

PRÁCTICA 04

Spinlocks y algunas Primitivas

Flores Linares Oscar Daniel 320208591
Márquez Corona Danna Lizette 320279991

05 de abril del 2026

1. Utiliza el programa *RunSpin* para probar cada uno de los spinlocks, cada integrante en el equipo debe completar la siguiente tabla: (**max-1** es el número máximo de hilos en tu compu menos uno)

Tabla Oscar: comparativa cuando la Tarea es muy breve								
SpinLocks / Counter	TAS	TTAS	Backoff	MCS	ALock	CLHLock	Reentrant	Counter Atomic
Tiempo (ms): 400 tareas con 4 hilos	6.5805	4.77	5.0706	4.9696	4.9630	4.8711	4.8286	4.9757
Tiempo (ms): 1000 tareas con 4 hilos	5.0676	4.9447	4.9542	4.7855	4.8193	5.0583	4.7851	4.7831
Tiempo (ms): 1000 tareas con 15 hilos	4.8954	4.8432	4.7637	4.8956	4.7698	4.8122	4.9482	5.0879

Tabla Danna: Comparativa cuando la Tarea es muy breve								
SpinLocks / Counter	TAS	TTAS	Backoff	MCS	ALock	CLHLock	Reentrant	Counter Atomic
Tiempo (ms): 400 tareas / 4 hilos	5.4197	2.6821	3.0869	5.2021	3.6702	3.2130	4.5005	4.3629
Tiempo (ms): 1000 tareas / 4 hilos	3.0301	3.2915	5.8508	3.7814	4.1467	3.5690	3.5016	3.5102
Tiempo (ms): 1000 tareas / 19 hilos	4.3253	7.3242	3.5204	4.4935	3.2459	3.2826	5.5680	8.2101

Cuadro 1: Resultados de ejecución obtenidos en un procesador de 14 núcleos (20 hilos lógicos).

2. Modificación del método `task(lock)`.

En lugar de incrementar un contador, modifica la función *task(lock)* para que los hilos trabajen con una matriz compartida de tamaño $n \times n$, donde $n = 10$. Cada hilo deberá asignar valores a la matriz dentro de una sección crítica protegida por *lock* y, una vez completada la asignación, imprimir su contenido. Implementa una función separada para la asignación y la impresión de la matriz. Cada integrante del equipo debe completar la siguiente tabla:

Tabla Oscar: cuando la Tarea es más pesada							
SpinLocks / Counter	TAS	TTAS	Backoff	MCS	ALock	CLHLock	Reentrant
Tiempo (ms): 100 tareas con 4 hilos	70.0429	67.4208	78.0911	deadlock	72.4719	deadlock	64.8997
Tiempo (ms): 100 tareas con 15 hilos	130.4921	135.2964	80.4667	deadlock	170.6087	deadlock	64.5634

Tabla Danna: cuando la Tarea es más pesada							
SpinLocks / Counter	TAS	TTAS	Backoff	MCS	ALock	CLHLock	Reentrant
Tiempo (ms): 100 tareas con 4 hilos	599.938	656.096	630.034	deadlock	634.5305999	deadlock	441.222
Tiempo (ms): 100 tareas con max-1 hilos	13388.3544	6820.9782	5660.7579	deadlock	5953.6851	deadlock	3646.68979

Durante la ejecución de cada lock, notamos que **MCSLock** y **CLHLock** nos dieron deadlock, esto sucedió porque en los códigos base de estos 2 algoritmos, hace falta el modificador *volatile* en las variables de espera. Por esto mismo al imprimir la matriz en la terminal con `System.out.print`, la sección crítica se vuelve muy lenta, lo que aumenta la contención y hace que los demás hilos giren en los ciclos `while(locked)`.

Como no son *volatile*, **JIT** guarda el valor en el caché del procesador y cuando el hilo anterior libera el candado, esto no llega a los demás hilos que están esperando en la memoria principal, haciendo que se queden atrapados en un ciclo infinito.

- Escojan al integrante del equipo con mayor número de hilos. ¿Cuáles son las 2 implementaciones más rápidas del inciso 1 ? ¿A qué crees se debe?

Basado en los datos de la tabla de Danna, las dos implementaciones más eficientes y consistentes son:

- ALock (Promedio: 3.6876 ms):** Aunque no fue el más rápido en el escenario de 4 hilos, fue el más veloz al escalar a 19 hilos (3.2459 ms), demostrando una estabilidad superior bajo alta contención.
- CLHLock (Promedio: 3.3549 ms):** Es el algoritmo más balanceado. Mantuvo tiempos bajos en las tres pruebas (3.21 ms, 3.56 ms y 3.28 ms), mostrando la menor variabilidad ante el cambio de carga y número de hilos.

La eficiencia de los algoritmos se puede entender mejor comparando el comportamiento de los hilos con una fila organizada frente a una multitud caótica. En los métodos simples como TAS y TTAS, todos los hilos (en este caso, hasta 19) intentan acceder a una misma variable al mismo tiempo. Esto genera un 'griterío' electrónico conocido como saturación del bus de datos; cada vez que un hilo pregunta si el candado está libre, distrae a todos los demás núcleos del procesador, desperdiciando tiempo y energía solo en la comunicación.

En otros algoritmos hay un caos ya que se satura el bus de datos, y desperdicia tiempo y energía para ver si un candado está libre, y en algoritmos como ALock y CLHLock asignan a cada hilo un turno específico en una fila virtual, así cada núcleo del procesador solo tiene que vigilar su propia 'bandera' o turno.

Además, el uso de una estructura de cola garantiza un orden de llegada FIFO, lo que asegura que el hilo que lleva más tiempo esperando sea el siguiente en entrar. Esto evita situaciones de 'atropellamiento' donde algunos hilos podrían quedarse esperando infinitamente mientras otros más rápidos les ganan el lugar.

4. Escojan al integrante con mayor número de hilos. ¿Cuáles son las 2 implementaciones más rápidas del inciso 2 ? ¿A qué crees se debe?

Tomando en cuenta los resultados de la tabla de Oscar los mejores resultados se dieron en las siguientes dos columnas.

- a) **ReentrantLock (promedio: 64.73 ms)**: Fue la columna que definitivamente tuvo los resultados más bajos en tiempo y el más estable comparado con los demás locks. Tanto para 4 hilos como para 15 los resultados se mantuvieron bastante estables, siendo ligeramente mejor con 15 hilos.
- b) **BackoffLock (promedio: 79.2789 ms)**: Fue el segundo algoritmo con mejores resultados y bastante estables con 4 hilos y 15 hilos, incluso distribuyéndose mejor con 4 hilos. A comparación del algoritmo que se mencionó anteriormente, este tuvo valores ligeramente más altos.

La principal razón por la que se dieron estos rendimientos es por el peso que dió la asignación e impresión de la matriz de 10×10 . Termina siendo una operación 'lenta', y hace que los hilos mantengan el candado bastante tiempo.

Sobre **ReentrantLock** no es un spinlock puro. Cuando un hilo no obtiene un candado, en vez de girar gastando la CPU, le pide al sistema que se suspenda hasta que el candado se libere. Y para tareas largas, el hecho de que no se desperdicie CPU esperando de forma activa, hace que sea más eficiente, pudiendo tener tiempos buenos y estables.

Sobre **BackoffLock**, a diferencia de **TAS** y **TTAS** que cayeron mucho para 15 hilos, este algoritmo funcionó porque tiene retrasos exponenciales de manera aleatoria. Cuando los hilos ven que la sección crítica tarda mucho en liberarse, estos se alejan y esperan más entre intentos, lo que hace que no haya mucho tráfico de memoria y haciendo que el hilo con el candado pueda imprimir su matriz sin ningún detenimiento de hardware.

5. ¿Cuáles son las implementaciones que satisfacen la propiedad de Justicia? (Por el momento ignora *ReentrantLock*). Argumenta como lo hacen.

Dentro de los algoritmos analizados, las implementaciones que cumplen estrictamente con la propiedad de justicia son **ALock**, **CLHLock** y **MCSLock**, ya que son "locks basados en cola" (*Queue-based locks*) los cuales evitan el problema de starvation, donde un hilo podría

quedarse esperando indefinidamente el recurso.

Como se explico en ejercicios anteriores, la forma en que logran esta justicia es mediante una estructura tipo FIFO. A diferencia de los algoritmos como TAS o TTAS, donde todos los hilos 'se amontonan' en la puerta y el sistema operativo elige al azar quién entra, los locks de cola asignan un turno específico a cada hilo. En el caso del **ALock**, esto se hace mediante un arreglo circular donde cada hilo toma un índice; en el **CLHLock** y **MCSLock**, se utiliza una lista ligada de nodos. El punto clave es que una vez que un hilo obtiene su lugar en la fila, nadie puede rebasarlo".